SOFTWARE REQUIREMENTS SPECIFICATION

for

MELODYBLEND

A Secure Full-Stack Social Music Discovery
and Playlist Blending Platform

Version 1.1

Prepared according to IEEE Std 830-1998

Prepared by : 1. Prashant Narayan Verma (1024160140)
2. Hazel Latka (1024160105)
3. Pridhi Thareja (1024160136)
4. Pranav Chauhan (1024160113)

Submitted to : Dr. Anushka

Course : UCS503 — Software Engineering

Department : Computer Science and Engineering

Institution : Thapar Institute of Engineering & Technology, Patiala

October 16, 2026

Revision History

Version	Date	Summary of changes
1.0	Oct 2026	Initial SRS.
1.1	Oct 2026	Technology stack made consistent (Next.js toolchain, Vite removed); roles of the iTunes Search API and the Spotify Web API defined; performance, reliability and scalability figures labelled as prototype targets or future goals; security wording aligned with Supabase Auth; cold-start weighting and formula notation defined; test statuses corrected; objectives, stakeholders, key features, diagrams, additional use cases and a traceability matrix added. Document restructured to follow the departmental SRS template.

Contents

Revision History i

1 Introduction 1

1.1 Purpose . . . 1
1.2 Intended Audience and Reading Suggestions . . . 1
1.3 Project Scope . . . 1
1.3.1 Scope . . . 1
1.3.2 Project Objectives . . . 2
1.4 Definitions, Acronyms and Abbreviations . . . 2
1.5 References . . . 3

2 Overall Description 3

2.1 Product Perspective . . . 3
2.2 User Classes and Characteristics . . . 5
2.3 Product Functions . . . 5
2.4 Operating Environment . . . 6
2.5 Design . . . 7
2.5.1 Use Case Model . . . 7
2.5.2 Data Flow Description . . . 12
2.5.3 Database Design . . . 16

3 System Features 19

3.1 Description and Priority . . . 19
3.2 Functional Requirements . . . 20
3.3 Core Algorithm Specifications . . . 23

3.3.1 Recommendation Score Formula . . . 23
3.3.2 Taste Compatibility Formula . . . 24
3.3.3 Blended Playlist Generation . . . 25

4 Other Nonfunctional Requirements 25
4.1 Performance Requirements . . . 25
4.2 Security Requirements . . . 26
4.3 Software Quality Attributes . . . 26
4.3.1 Usability (NFR-U) . . . 27
4.3.2 Reliability (NFR-R) . . . 27
4.3.3 Maintainability (NFR-M) . . . 27
4.3.4 Scalability (NFR-SC) — Architectural Goals / Future Considerations . . . 27
4.4 Business Rules . . . 28
4.5 External Interface Requirements . . . 28
4.6 Role of the External Music APIs . . . 29
4.7 Design and Implementation Constraints . . . 29
4.8 Assumptions and Dependencies . . . 29

5 Other Requirements 30

6 Verification and Validation 30
6.1 Testing Strategy . . . 30
6.2 Initial Test Cases . . . 31

A Technology Stack Summary 32

B Development Phase Plan 33

C Team Responsibilities 33

D Known Limitations 33

E Requirements Traceability Matrix 34

List of Figures

2.1 System architecture of MelodyBlend 4
2.2 Types of users 5
2.3 Use case diagram of MelodyBlend 7
2.4 Sequence diagram for the Music Blend session (UC-02) 9
2.5 DFD Level 0 — context diagram 13
2.6 DFD Level 1 of MelodyBlend 15
2.7 Entity Relationship Diagram of MelodyBlend (crow's foot = many side) 17

List of Tables

2.3 Product functions 6
2.6 DFD Level 1 — major processes 13
2.7 Database entity description 18
3.1 Feature modules and requirement priorities 19

1 Introduction

1.1 Purpose

Modern music-streaming services are largely *individually centric*: they recommend music to one listener at a time, give little explanation for their suggestions, and offer very limited ways for friends to combine their tastes. **MelodyBlend** is a secure, full-stack, browser-based social music discovery and playlist blending platform that addresses these gaps. A user builds a preference profile (genres, moods, musical eras and favourite artists) and receives *transparent, weighted* song recommendations; two users can *blend* their tastes through a short invitation code and obtain a compatibility score together with a balanced shared playlist.

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of MelodyBlend. It is prepared in accordance with the IEEE 830-1998 standard and follows the section structure prescribed in the departmental SRS template.

1.2 Intended Audience and Reading Suggestions

This SRS is written for the development team, the project supervisor, testers, future maintainers and other stakeholders who review the software engineering process. It describes the internal, external, functional and non-functional aspects of MelodyBlend. Performance, reliability and scalability figures in this document are **prototype evaluation targets or architectural goals** unless explicitly stated otherwise (see Section 4 and Appendix D).

Reader	Suggested reading
Project supervisor	Sections 1–2 for scope and context, Section 3 for requirements, Section 6 and Appendix E for verification and traceability.
Developers	Section 2.5 (design models), Section 3 (functional requirements and algorithms), Section 4 (constraints and quality targets).
Testers	Section 3, Section 6 (test strategy and test cases) and Appendix E.
Maintainers	Section 2.5.4 (database design), Section 5 and Appendix D (known limitations).

1.3 Project Scope

1.3.1 Scope

MelodyBlend is a browser-based full-stack platform that enables:

- **Personalized music discovery** — users build preference profiles and receive transparent, weighted song recommendations.
- **Collaborative playlist blending** — two users blend their musical tastes using a mathematical compatibility algorithm and receive a proportional shared playlist.
- **Social music interaction** — listening streaks, peer-to-peer song sharing (Song Snaps), playlist organisation and friend connections.
- **Administration** — role-restricted catalogue curation, user management and content moderation.

Technology summary. The front end is built with Next.js (React 19, TypeScript) and Tailwind CSS. Supabase provides authentication, real-time messaging, file storage and a PostgreSQL database. The application is deployed on Vercel. Music catalogue metadata, album artwork and 30-second

preview URLs are obtained from the **iTunes Search API**; the **Spotify Web API** is used only for additional artist/track metadata enrichment. MelodyBlend does **not** provide full-length Spotify streaming.

Out of scope: full-length licensed streaming, multi-user (> 2) blends and native mobile applications (all identified as future work).

1.3.2 Project Objectives

ID	Objective	Related requirements
O1	Design and implement a responsive full-stack web application for social music discovery, search and 30-second preview playback.	**FR-12–FR-16**
O2	Provide secure registration, login and profile management using Supabase Auth.	**FR-01–FR-06**
O3	Allow users to create and maintain preference profiles across genres, moods, eras and favourite artists.	**FR-07–FR-11**
O4	Implement a transparent, deterministic, weighted recommendation engine that explains each recommendation.	**FR-17–FR-20**, **FR-41**
O5	Provide a collaborative Music Blend feature that computes a taste-compatibility score and generates a balanced shared playlist.	**FR-28–FR-35**
O6	Provide engagement tools: playlists, listening streaks and history, friend connections and Song Snap sharing.	**FR-21–FR-27**, **FR-36–FR-40**
O7	Provide protected administrative tools for catalogue curation, user management and content moderation.	**FR-42–FR-44**

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token — signed token issued by Supabase Auth to represent an authenticated session
RLS	Row Level Security — PostgreSQL per-row access control
SSR / CSR	Server-Side Rendering / Client-Side Rendering (both supported by Next.js)
DFD / ERD	Data Flow Diagram / Entity Relationship Diagram
Blend	Collaborative session in which two users generate a shared music playlist (stored in `blend_sessions`)
Compatibility Score	Numeric percentage (0–100%) $C(u_A, u_B)$ representing the musical taste similarity of two users
Recommendation Score	Weighted score $S(u, s) \in [0, 1]$ computed for each song s against the preferences of user u

Term / Acronym	Definition
Cold-start user	A user with no rows in `listening_history`; $M_{history}$ is excluded and the remaining weights are renormalised (Section 3.3)
Song Snap	Peer-to-peer music sharing feature between connected users
Listening Streak	Consecutive days with at least one 30-second preview played
Jaccard Similarity	Set similarity $\lvert A \cap B \rvert$ / $\lvert A \cup B \rvert$
30-second preview	Short audio clip supplied by the iTunes Search API; the only audio MelodyBlend plays
Prototype / evaluation target	A measurable goal used to evaluate the prototype; **not** a guaranteed production capability
Supabase	Managed backend platform providing Auth, PostgreSQL, Realtime and Storage
TIET	Thapar Institute of Engineering and Technology

1.5 References

[1] IEEE. *IEEE Recommended Practice for Software Requirements Specifications*. Std 830-1998.
[2] Roger S. Pressman. *Software Engineering: A Practitioner's Approach*. 7th ed., McGraw-Hill, 2010.
[3] Ian Sommerville. *Software Engineering*. 9th ed., Addison-Wesley, 2011.
[4] Spotify AB. *Spotify Web API Documentation*. 2024. `https://developer.spotify.com/documentation/web-api/`
[5] Supabase Inc. *Supabase Platform Documentation*. 2024. `https://supabase.com/docs`
[6] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. `https://www.postgresql.org/docs/`
[7] Apple Inc. *iTunes Search API Documentation*. `https://developer.apple.com/library/archive/documentation/AudioVideo/Conceptual/iTuneSearchAPI/`
[8] Vercel Inc. *Next.js Documentation*. `https://nextjs.org/docs`

2 Overall Description

2.1 Product Perspective

MelodyBlend is a new, independent product that follows a three-tier architecture (Figure 2.1). The technology stack is defined once (Appendix A) and used identically throughout this document.

- **Presentation layer:** Next.js (App Router) + React + TypeScript web application with responsive UI components styled with Tailwind CSS and a persistent 30-second preview player.
- **Application layer:** recommendation engine, Music Blend generator, input validation and external music-API calls, implemented as isolated TypeScript modules executed in the browser or in Next.js server code (Route Handlers / Server Actions) on Vercel.
- **Data/backend layer:** Supabase Auth (signed JWT sessions), PostgreSQL 15+ with RLS, Supabase Realtime and Storage.
- **External music services:** iTunes Search API (catalogue metadata, artwork, 30-second preview URLs) and Spotify Web API (additional artist/track metadata enrichment only).

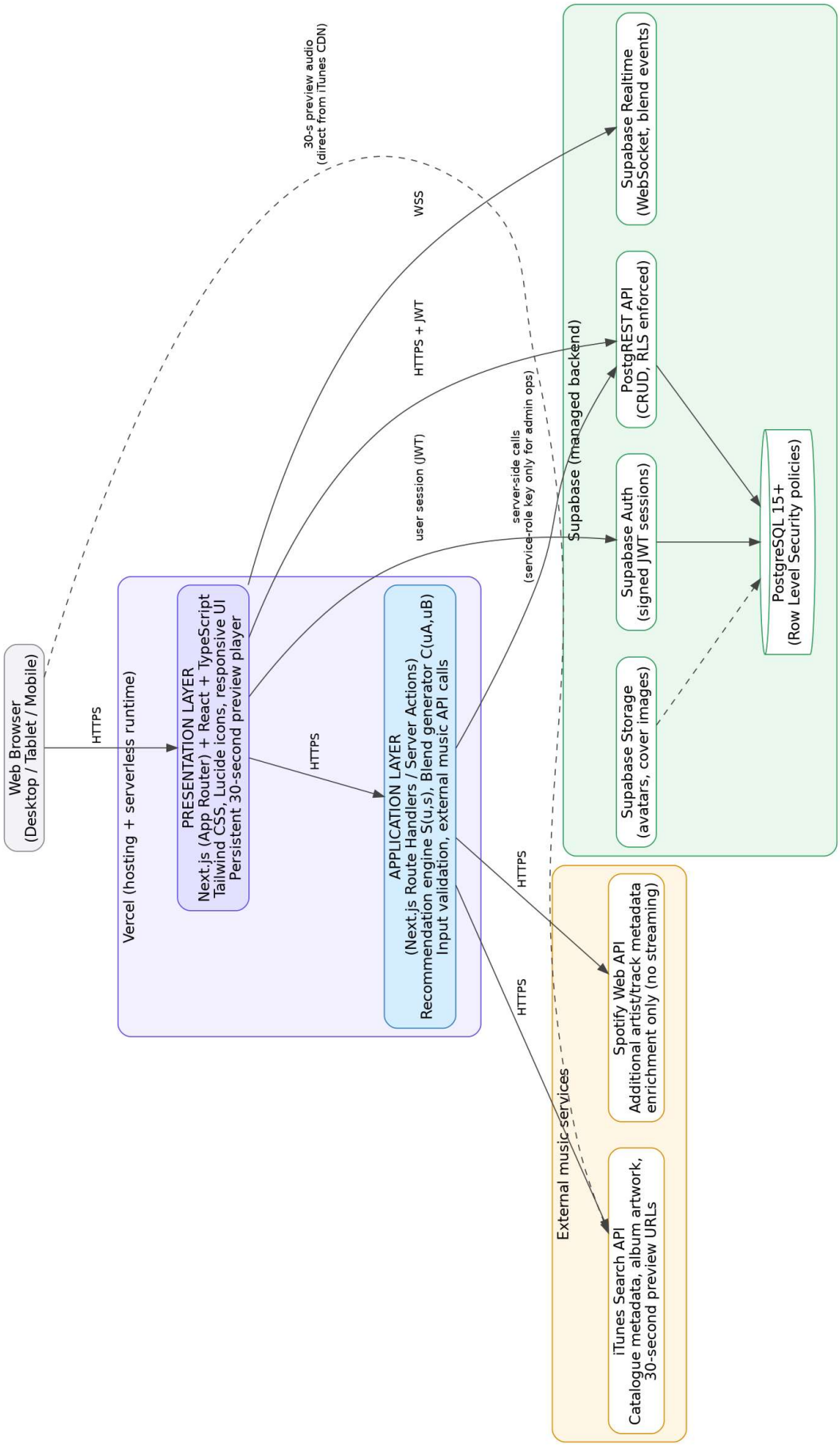


Figure 2.1: System architecture of MelodyBlend

2.2 User Classes and Characteristics

MelodyBlend has four classes of users (Figure 2.2).

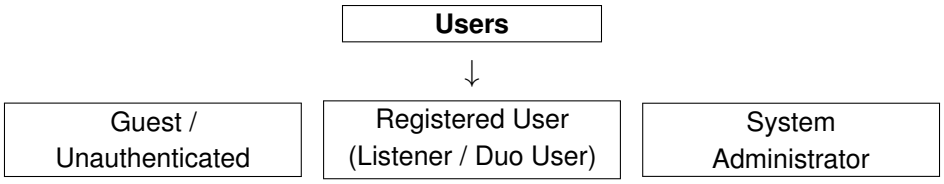


Figure 2.2: *Types of users*

User class	Description
Individual Listener (primary)	Personal music discovery and playlist management user. Low-to-moderate technical proficiency. Daily-to-weekly use.
Social / Duo User (primary)	Uses the Music Blend feature with a friend or partner. Situational use (road trips, parties). Low-to-moderate proficiency.
Guest / Unauthenticated	Visitor limited to the landing page, login and registration.
System Administrator	Holds a distinct Administrator role (separate from normal users). Manages the music catalogue, users and reported content through protected admin operations. High proficiency, periodic use.

Stakeholders

Stakeholder	Role	Interest in the system
End users (listeners, duos)	Primary users	Accurate, explainable recommendations; easy blending with friends; privacy of their data.
Administrators / curators	Operators	Catalogue integrity, moderation of reported content, platform health.
Development team (4 students)	Designers, developers, testers	Deliver a correct, well-documented, testable prototype within the 12-week plan.
Project supervisor (Dr. Anushka)	Evaluator	Software engineering process quality, requirement clarity and traceability.
Future maintainers	Maintainers	Modular code, version-controlled schema, clear requirements.
Apple (iTunes Search API) and Spotify	External service providers	Compliance with API terms: 30-second previews only, rate limits, no full-track streaming.
Supabase and Vercel	Infrastructure providers	Hosting, authentication, database and deployment within plan quotas.

2.3 Product Functions

MelodyBlend groups its capabilities into ten functions, summarised in Table 2.3. Before using any function other than browsing the landing page, a user has to register and log in.

Table 2.3: Product functions

#	Function	Summary	Requirements
1	User Account Management	Register, authenticate, manage profile and avatar	**FR-01–FR-06**
2	Preference Profiling	Select and update genres, moods, musical eras, favourite artists	**FR-07–FR-11**
3	Music Discovery	Browse catalogue, search, filter, preview audio	**FR-12–FR-16**
4	Recommendation Engine	Receive personalized weighted song recommendations with explanations	**FR-17–FR-20**
5	Playlist Management	Create, edit, delete and share playlists	**FR-21–FR-27**
6	Music Blend	Invite codes, compatibility scores, blended playlists	**FR-28–FR-35**
7	Social Features	Friend connections, Song Snap sharing, social feed	**FR-36–FR-38**
8	Listening Streaks & History	Track daily streaks and playback history	**FR-39–FR-41**
9	Audio Playback	In-platform 30-second audio preview player (iTunes preview URLs)	**FR-15**
10	Administration	Catalogue curation, user management, content moderation (admin role only)	**FR-42–FR-44**

Key features

Key feature	Description
Transparent recommendations	Deterministic weighted score $S(u, s)$ with a human-readable explanation for each recommended song.
Music Blend	Two users blend tastes via an invitation code; the system returns a compatibility score and a 20-track blended playlist (8 / 8 / 4).
30-second preview player	Persistent mini-player across routes playing iTunes 30-second previews (no full-length streaming).
Listening streaks and history	Every preview play is logged; consecutive-day streaks are displayed and feed the $M_{history}$ signal.
Social sharing	Friend connections and Song Snaps with notifications.
Secure by design	Supabase Auth sessions, Row Level Security on user data, server-only privileged credentials, role-restricted admin operations.

2.4 Operating Environment

Attribute	Specification
Client platform	Chrome $\geq$ 110, Firefox $\geq$ 115, Safari $\geq$ 16, Edge $\geq$ 110
Device support	Desktop ($\geq$1024 px), tablet (768–1023 px), mobile (320–767 px)
Internet	Active connection required (API calls, 30-second preview playback)
Backend hosting	Supabase Cloud (managed PostgreSQL 15+, Auth, Realtime, Storage)

Attribute	Specification
Front-end hosting	Vercel (Next.js deployment, including serverless Route Handlers)
OS support	Cross-platform via browser (Windows, macOS, Linux, Android, iOS)
Node.js version	$\geq$ 18.18 LTS (development environment; required by Next.js)

2.5 Design

This section presents the design models of MelodyBlend: the use cases, the Music Blend interaction sequence, the data flow diagrams and the database design.

2.5.1 Use Case Model

Figure 2.3 shows the use case diagram. Use cases UC-01 to UC-10 together cover all functional modules (traceability in Appendix E).

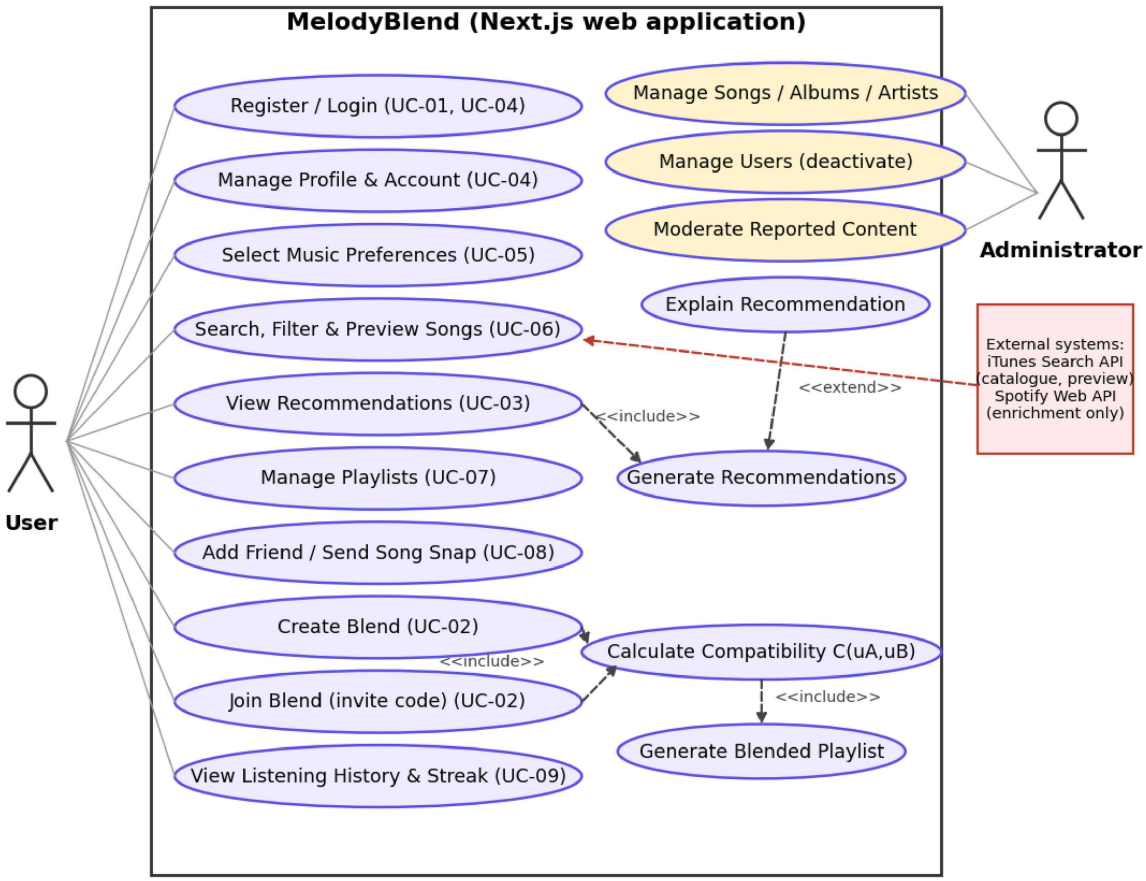


Figure 2.3: *Use case diagram of MelodyBlend*

User Registration (UC-01)

Use Case ID	UC-01
Name	User Registration
Actor	Guest / Unauthenticated Visitor
Precondition	User has not registered on the platform.
Trigger	User navigates to the Sign-Up page.
Main flow	1. User enters email, password and username. 2. System validates input format and password strength. 3. System calls Supabase Auth to create the account. 4. A verification email is sent. 5. A profile record (`profiles`) is created. 6. User is redirected to preference-profiling onboarding.
Alternate flow	If the email already exists: "An account with this email already exists."
Postcondition	New user account and profile record created. User begins preference onboarding.
Related FR	**FR-01**, **FR-07**, **FR-08**, **FR-09**

Music Blend Session (UC-02)

Use Case ID	UC-02
Name	Music Blend Session
Actors	User A (Initiator), User B (Joiner)
Precondition	Both users are registered and have completed preference profiling.
Trigger	User A clicks "Start a Blend" on the Blends Hub.
Main flow	1. System generates a unique 6-character invite code and stores the session in `blend_sessions` (status *pending*). 2. User A shares the code with User B. 3. User B enters the code on the Join Blend screen. 4. System validates the code and links both profiles. 5. System computes the Jaccard-based compatibility score $C(u_A, u_B)$. 6. System generates the 20-track blended playlist. 7. Both users see the compatibility score and playlist (Supabase Realtime notifies User A). 8. Users can save or share the playlist.
Alternate flows	A1: Invalid or expired code $\rightarrow$ "Invalid or expired invitation code." A2: User A has no preferences $\rightarrow$ prompt to complete profiling. A3: User selects a pre-built blend preset (**FR-34**). A4: User opens Blend History (**FR-35**).
Postcondition	Compatibility score and blended playlist generated; optionally saved.
Related FR	**FR-28**–**FR-35**

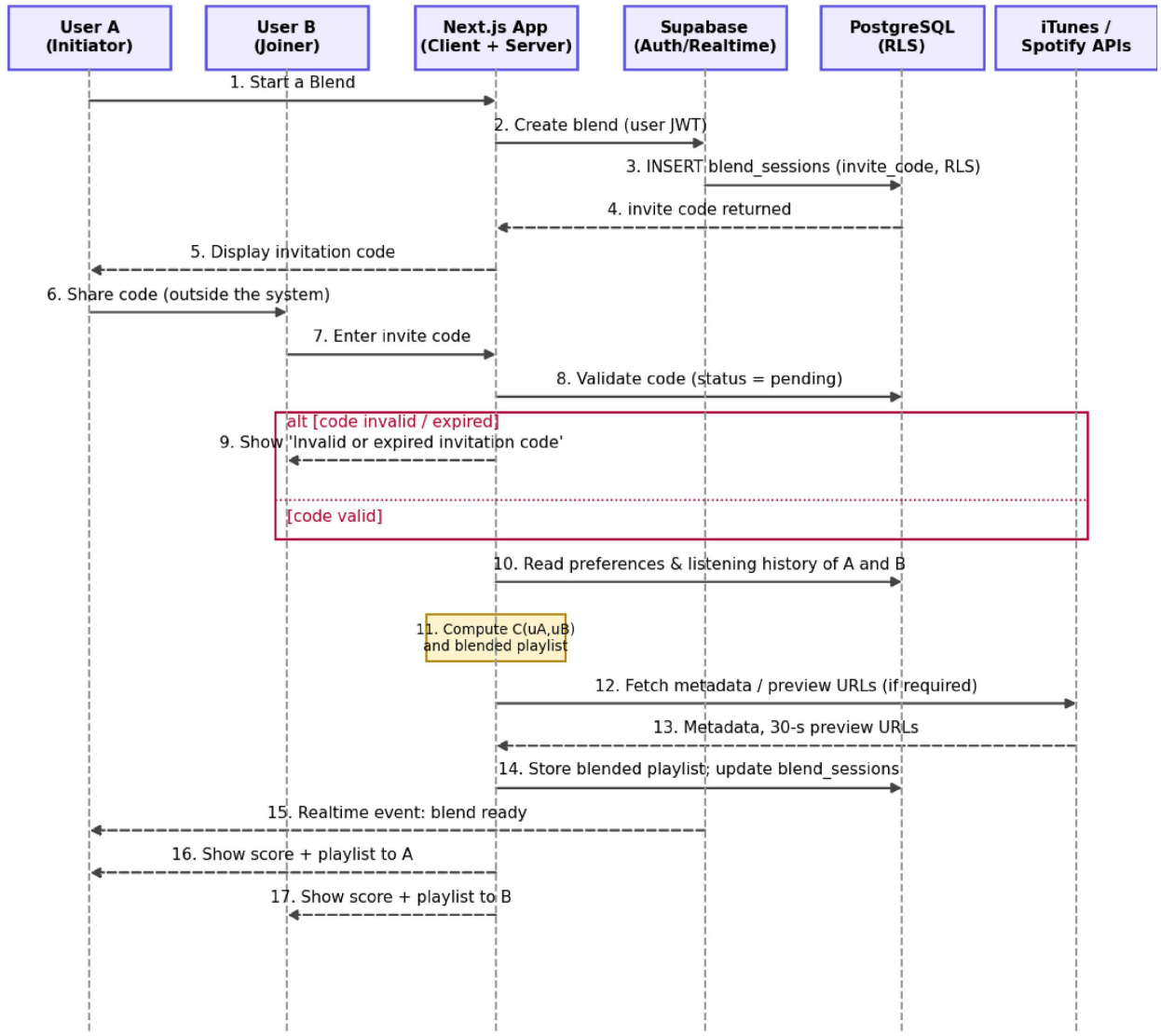


Figure 2.4: *Sequence diagram for the Music Blend session (UC-02)*

Personalized Song Recommendation (UC-03)

Use Case ID	UC-03
Name	Personalized Song Recommendation
Actor	Authenticated Individual Listener
Precondition	User is logged in. At least genres, moods or eras are set in the preference profile.
Trigger	User navigates to the Discovery / Recommendations page.
Main flow	1. System retrieves user preferences and listening history from PostgreSQL. 2. System applies the weighted score $S(u,s)$ across the song catalogue. 3. System returns the top-20 highest-scoring songs ranked by score. 4. Each song displays a score breakdown (genre match, mood match, etc.).
Alternate flow	Cold start (no listening history) $\rightarrow$ $M_{history}$ is excluded and the remaining weights are renormalised by $1/0.85$ (Section 3.3).
Postcondition	Ranked list of personalized recommendations displayed.
Related FR	**FR-17**, **FR-18**, **FR-19**, **FR-20**, **FR-41**

Authenticate and Manage Account (UC-04)

Use Case ID	UC-04
Name	Authenticate and Manage Account
Actor	Registered User
Precondition	User has a registered account.
Main flow	1. User signs in with email/password (Supabase Auth). 2. Session JWT is issued and silently refreshed. 3. User may edit username, bio and avatar, request a password-reset link, log out, or request account deletion.
Alternate flow	Invalid credentials $\rightarrow$ descriptive error, no session created. Expired reset link $\rightarrow$ request a new link.
Related FR	**FR-02**, **FR-03**, **FR-04**, **FR-05**, **FR-06**

Manage Music Preferences (UC-05)

Use Case ID	UC-05
Name	Manage Music Preferences
Actor	Authenticated User
Precondition	User is logged in.
Main flow	1. User selects genres, moods and eras. 2. User bookmarks/unbookmarks favourite artists. 3. System stores them (`user_preferences`, `user_favourite_artists`). 4. Future recommendations use the updated preferences.
Alternate flow	No selection $\rightarrow$ prompt to choose at least one preference.
Related FR	**FR-07**–**FR-11**

Discover and Preview Music (UC-06)

Use Case ID	UC-06
Name	Discover and Preview Music
Actor	Authenticated User; iTunes Search API (external); Spotify Web API (external, enrichment only)
Precondition	User is logged in.
Main flow	1. User browses, searches or filters the catalogue. 2. System returns catalogue data (iTunes metadata and artwork; optional Spotify enrichment). 3. User plays a 30-second preview in the mini-player. 4. User may bookmark a song.
Alternate flow	External API throttled (HTTP 429) → retry with backoff (NFR-R5), then show an error or cached/seed data. Spotify unavailable → continue without enrichment.
Related FR	**FR-12–FR-16**

Manage Playlists (UC-07)

Use Case ID	UC-07
Name	Manage Playlists
Actor	Authenticated User
Precondition	User is logged in.
Main flow	1. User creates a playlist and sets visibility. 2. User adds, removes and reorders songs. 3. User may delete the playlist after confirmation. 4. Other users can view public playlists.
Alternate flow	Private playlist requested by non-owner → access denied by RLS.
Related FR	**FR-21–FR-27**

Friends and Song Snaps (UC-08)

Use Case ID	UC-08
Name	Friends and Song Snaps
Actor	Authenticated User (sender), Authenticated User (receiver)
Precondition	Both users are registered.
Main flow	1. User sends a friend request; the other user accepts (`friendship`). 2. User sends a song as a Song Snap (`song_snap`). 3. Receiver is notified and can play the 30-second preview. 4. Users view the social feed.
Alternate flow	Request to a non-friend → Song Snap rejected.
Related FR	**FR-36–FR-38**

Listening History and Streak (UC-09)

Use Case ID	UC-09
Name	Listening History and Streak
Actor	Authenticated User
Precondition	User is logged in.
Main flow	1. Each preview playback is logged in `listening_history`. 2. System updates `listening_streaks` (consecutive days). 3. User views history and current streak. 4. History feeds the $M_{history}$ signal.
Alternate flow	No play on a day → streak resets on the next play.
Related FR	**FR-39**, **FR-40**, **FR-41**

Administer Platform (UC-10)

Use Case ID	UC-10
Name	Administer Platform
Actor	Administrator
Precondition	User holds the Administrator role (server-controlled claim).
Main flow	1. Administrator opens the admin interface (role verified server-side). 2. Administrator adds, edits or removes songs, albums and artists. 3. Administrator deactivates policy-violating accounts or removes reported playlists/Song Snaps via server-side handlers.
Alternate flow	Non-admin user attempts access → request denied; the service-role key is never exposed to the client.
Related FR	**FR-42**, **FR-43**, **FR-44**

2.5.2 Data Flow Description

Level 0 — Context diagram. The MelodyBlend system boundary (Figure 2.5) interacts with five external entities:

- **Registered User** — provides login credentials, preference updates, search, playlist, blend and playback interactions; receives recommendations, playlists, compatibility scores and notifications.
- **Administrator** — provides catalogue updates and user/content moderation actions; receives reports and moderation items.
- **iTunes Search API** — provides catalogue metadata, album artwork and 30-second preview URLs.
- **Spotify Web API** — provides additional artist/track metadata for enrichment only.
- **Email Service (Supabase SMTP)** — receives requests to send verification and password-reset emails.

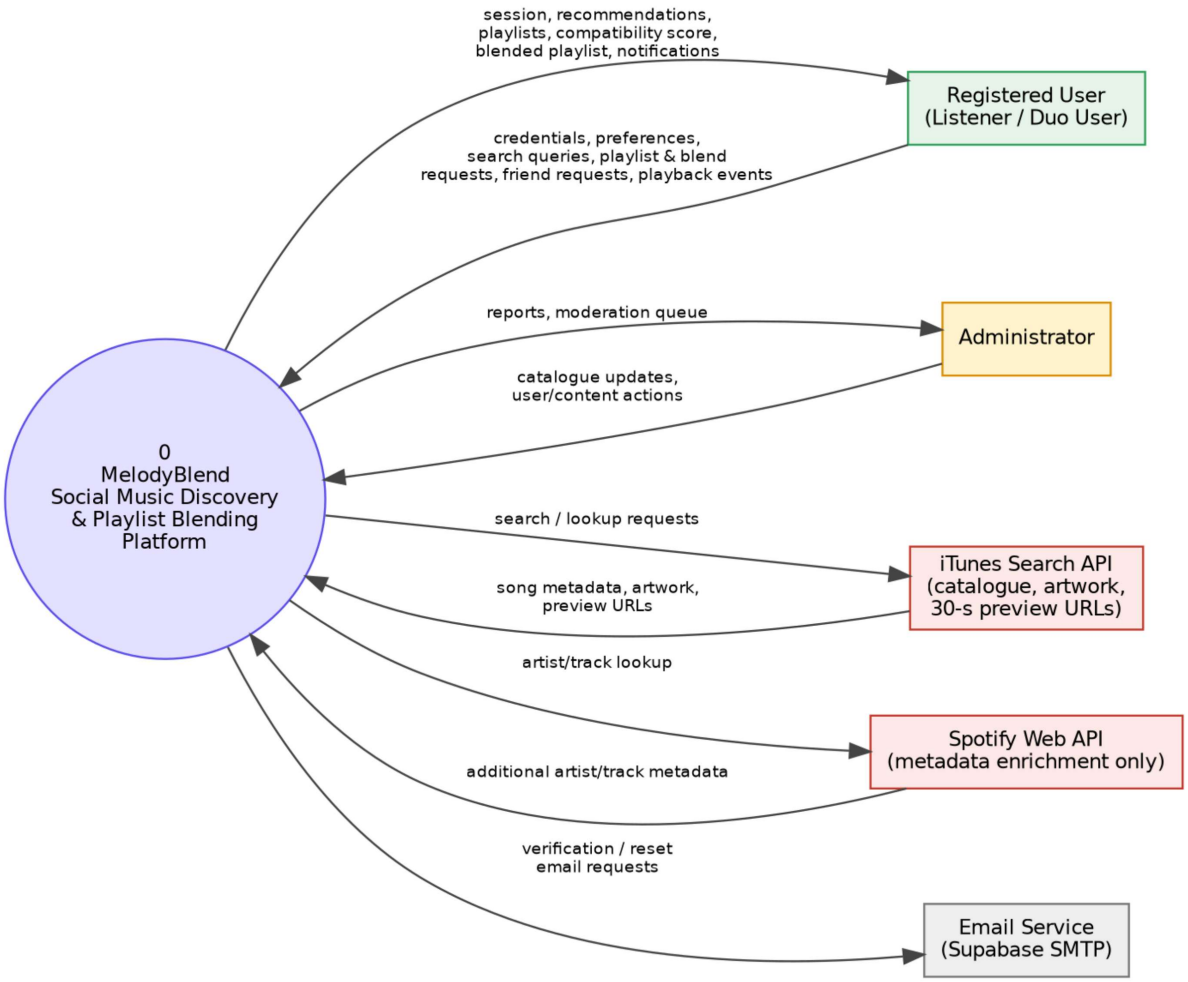


Figure 2.5: *DFD Level 0 — context diagram*

Level 1 — Major processes. Figure 2.6 (landscape page) shows the Level 1 decomposition; Table 2.6 lists the processes, data and requirement links.

Table 2.6: *DFD Level 1 — major processes*

Process	Inputs	Outputs	Data store (tables)	FRs
P1: Authentication	User credentials	JWT token, session	D1: `profiles`, `auth.users`	**FR-01–FR-06**
P2: Preference Profiling	Genre/mood/era selections, favourite artists	Stored preferences	D2: `user_preferences`, `user_favourite_artists`, `user_favourite_songs`	**FR-07–FR-11**
P3: Music Catalogue	Search query, filter parameters; iTunes/Spotify data	Song/artist/album results, previews	D3: `songs`, `albums`, `artists`, `genres`, `moods`	**FR-12–FR-16**
P4: Recommendation Engine	User preferences + history	Ranked song list with explanations	D3, D2, D7	**FR-17–FR-20**, **FR-41**
P5: Playlist Management	Song IDs, playlist metadata	Playlist records	D4: `playlists`, `playlist_songs`	**FR-21–FR-27**

Process	Inputs	Outputs	Data store (tables)	FRs
P6: Blend Processing	Invite code, user preferences and history	Compatibility score, blended playlist	D5: `blend_sessions`; D4; D2; D7	**FR-28–FR-35**
P7: Social Layer	Friend requests, Song Snaps	Friend links, shared songs	D6: `friendship`, `song_snap`	**FR-36–FR-38**
P8: History & Streaks	Playback events	History log, streak count	D7: `listening_history`, `listening_streaks`	**FR-39–FR-41**
P9: Administration	Admin actions on catalogue, users, content	Updated catalogue, deactivated accounts, removed content	D3, D1, D4, D6	**FR-42–FR-44**

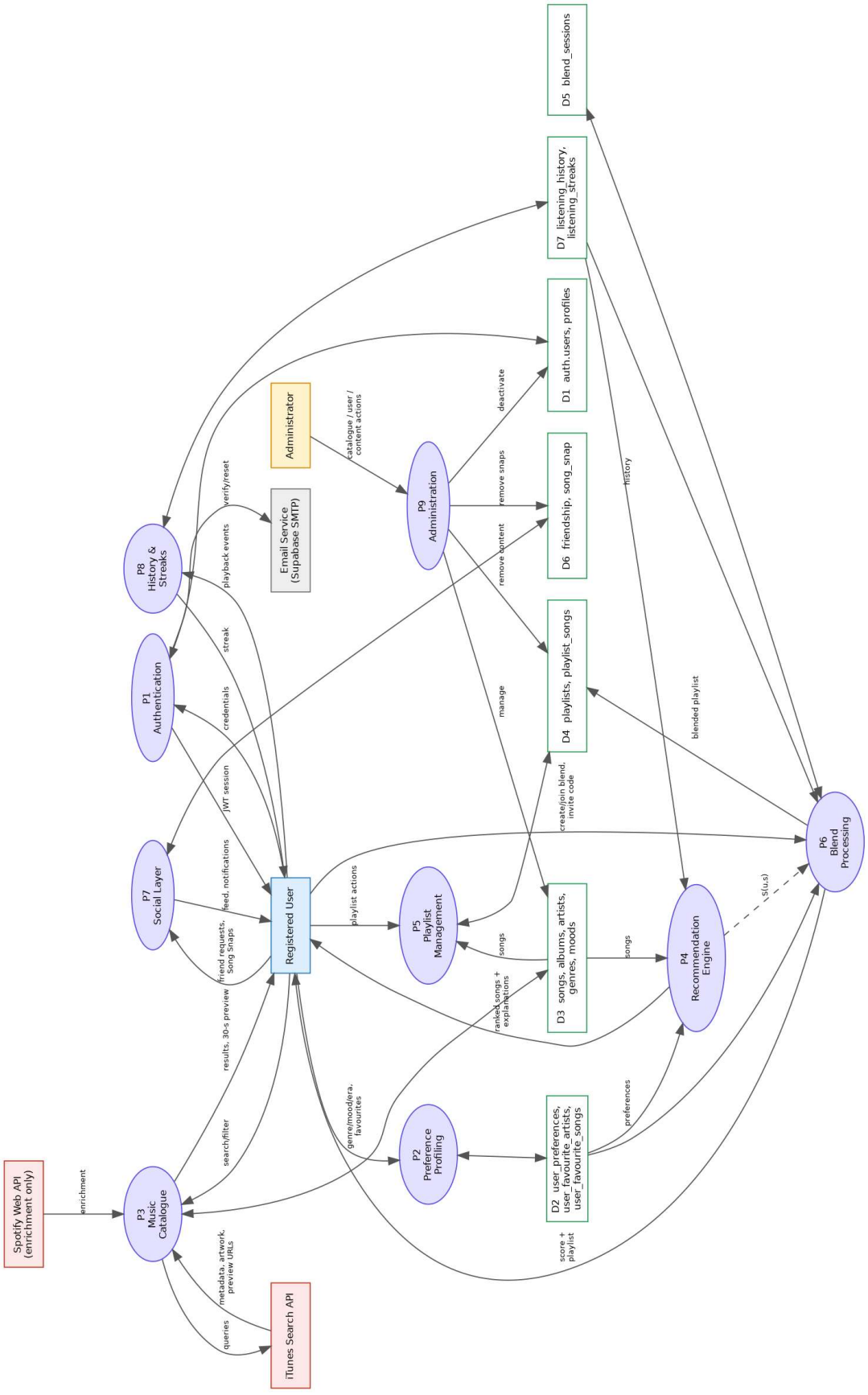


Figure 2.6: DFD Level 1 of MelodyBlend

2.5.3 Database Design

The relational schema is designed in Third Normal Form (3NF). Figure 2.7 shows the entity relationship diagram and Table 2.7 describes each entity.

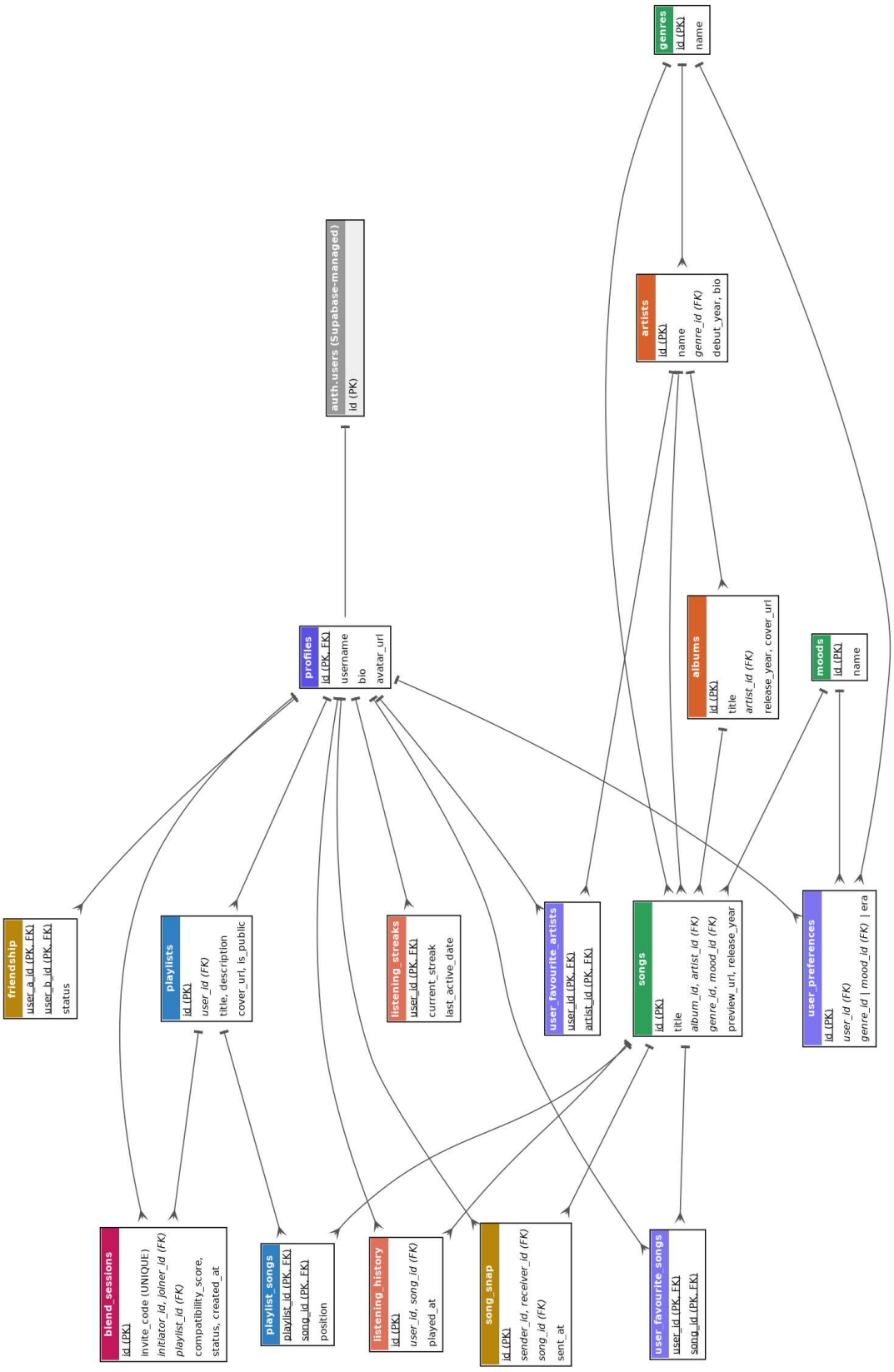


Figure 2.7: Entity Relationship Diagram of MelodyBlend (crow's foot = many side)

Table 2.7: *Database entity description*

Entity	Table	Key attributes	Purpose	FR / DFD
User Profile	`profiles`	id (PK, FK→auth.users), username, bio, avatar_url	User identity and profile metadata	**FR-04**; P1
Artist	`artists`	id (PK), name, genre_id (FK), debut_year, bio	Music artist catalogue	**FR-12**, **FR-42**; P3, P9
Album	`albums`	id (PK), title, artist_id (FK), release_year, cover_url	Musical release groupings	**FR-12**, **FR-42**; P3, P9
Song	`songs`	id (PK), title, album_id (FK), artist_id (FK), genre_id (FK), mood_id (FK), preview_url, release_year	Core track catalogue	**FR-12**–**FR-15**, **FR-17**; P3, P4
Genre	`genres`	id (PK), name	Standardised genre taxonomy	**FR-07**, **FR-14**; P2
Mood	`moods`	id (PK), name	Standardised mood taxonomy	**FR-08**, **FR-14**; P2
User Preferences	`user_preferences`	id (PK), user_id (FK), genre_id (FK, nullable), mood_id (FK, nullable), era (nullable)	Multi-valued preference profile: each row holds exactly one preference value (CHECK: exactly one of genre_id, mood_id, era is non-null)	**FR-07**–**FR-09**, **FR-11**; P2
Favourite Songs	`user_favourite_songs`	user_id (PK, FK), song_id (PK, FK)	Bookmarked songs	**FR-16**; P2/P3
Favourite Artists	`user_favourite_artists`	user_id (PK, FK), artist_id (PK, FK)	Bookmarked artists (M_{artist} signal)	**FR-10**; P2
Playlist	`playlists`	id (PK), user_id (FK), title, description, cover_url, is_public	User-curated and saved blended track collections	**FR-21**–**FR-27**, **FR-33**; P5
Playlist Songs	`playlist_songs`	playlist_id (PK, FK), song_id (PK, FK), position	Bridge table (M:N playlist–song)	**FR-23**–**FR-25**; P5
Listening History	`listening_history`	id (PK), user_id (FK), song_id (FK), played_at	Timestamped playback log ($M_{history}$ signal)	**FR-39**, **FR-41**; P8
Listening Streaks	`listening_streaks`	user_id (PK, FK), current_streak, last_active_date	Daily streak tracker	**FR-40**; P8
Friendship	`friendship`	user_a_id (PK, FK), user_b_id (PK, FK), status	Social connection graph	**FR-36**; P7
Song Snap	`song_snap`	id (PK), sender_id (FK), receiver_id (FK), song_id (FK), sent_at	Peer-to-peer song sharing	**FR-37**, **FR-38**; P7
Blend Session	`blend_sessions`	id (PK), invite_code (UNIQUE), initiator_id (FK), joiner_id (FK, nullable), playlist_id (FK, nullable), compatibility_score, status, created_at	Stores invite code, participants, score and result; supports validation (**FR-30**) and history (**FR-35**)	**FR-28**–**FR-35**; P6

Naming notes. Table names use the British spelling "favourite"; the prose uses the same spelling for that

concept. "Username" (`profiles.username`) is the user's display name. Blend status values are pending, active, completed *and* expired *(validity period configurable). All user-owned child rows use ON DELETE CASCADE (**FR-06**).*

Algorithm-to-schema references.

Symbol	Derived from
M_{genre}	`songs.genre_id` compared with `user_preferences.genre_id` (user u)
M_{mood}	`songs.mood_id` compared with `user_preferences.mood_id`
M_{artist}	`songs.artist_id` compared with `user_favourite_artists.artist_id`
M_{era}	decade of `songs.release_year` compared with `user_preferences.era` (stored as a decade, e.g. 1990)
$M_{history}$	play counts from `listening_history` (user_id, song_id)
G, MD, E, H (sets)	genre IDs, mood IDs and eras from `user_preferences`; distinct song IDs from `listening_history`

3 System Features

MelodyBlend is a social music discovery and playlist-blending web application. Its main purpose is to turn user preferences into transparent recommendations and to let two users blend their tastes into a shared playlist.

3.1 Description and Priority

All features are necessary for the complete product, but they differ in importance. The modules below are listed from highest to lowest importance to the project goals. Requirement priorities are H = High, M = Medium and L = Low.

Table 3.1: *Feature modules and requirement priorities*

#	Feature module	Description	H	M	L
1	Music Blend (core feature)	Invite code, compatibility score and 20-track blended playlist. **Module 6.**	5	1	2
2	Recommendation Engine	Transparent, deterministic weighted recommendations. **Module 4.**	2	2	0
3	User Account Management	Registration, login, sessions, profile. **Module 1.**	3	2	1
4	Preference Profiling	Genres, moods, eras, favourite artists. **Module 2.**	4	1	0
5	Music Discovery & Catalogue	Browse, search, filter, 30-second preview. **Module 3.**	3	2	0
6	Playlist Management	Create, edit, share playlists. **Module 5.**	3	4	0
7	Listening History & Streaks	Playback log and daily streaks. **Module 8.**	2	1	0
8	Social Features	Friends, Song Snaps, feed. **Module 7.**	0	2	1

#	Feature module	Description	H	M	L
9	Administration	Catalogue, user and content moderation. **Module 9.**	0	1	2
		Total (44 functional requirements)	**22**	**16**	**6**

3.2 Functional Requirements

The website is built with the technology stack described in Appendix A: **Front end** — Next.js, React, TypeScript, Tailwind CSS; **Back end / services** — Supabase (Auth, PostgREST, Realtime, Storage) and Next.js Route Handlers on Vercel; **Database** — PostgreSQL 15+ with Row Level Security. Requirements are grouped by module. Priority: H = High, M = Medium, L = Low.

Module 1: User Account Management

Req ID	Requirement	Description	Pri
FR-01	User Registration	The system shall allow new users to register through Supabase Auth with a valid email address and a password ($\geq$ 8 characters, mixed case). A verification email shall be sent.	H
FR-02	User Authentication	The system shall authenticate registered users via email/password through Supabase Auth, which issues a signed JWT session token on success.	H
FR-03	Session Management	Sessions shall use Supabase JWT access tokens with automatic silent refresh by the Supabase client. Sessions shall be invalidated on explicit logout.	H
FR-04	Profile Management	Authenticated users shall update username, bio (max 200 characters) and avatar image.	M
FR-05	Password Reset	The system shall provide password reset via an email-delivered one-time link (Supabase Auth).	M
FR-06	Account Deletion	Users shall be able to request account deletion with cascade-delete of all personal data. The privileged deletion step is executed server-side.	L

Module 2: Preference Profiling

Req ID	Requirement	Description	Pri
FR-07	Genre Selection	Users shall select one or more preferred genres from a pre-defined taxonomy stored in the `genres` table (e.g., Pop, Rock, Jazz, Hip-Hop, Classical, Electronic, R&B, Country, Metal, Indie, Bollywood, Punjabi, Lo-Fi).	H
FR-08	Mood Selection	Users shall select one or more preferred moods from the `moods` table (e.g., Happy, Sad, Energetic, Chill, Romantic, Focused, Melancholic, Hype).	H
FR-09	Era Selection	Users shall select preferred musical decades (1960s through 2020s).	H

Req ID	Requirement	Description	Pri
FR-10	Artist Bookmarking	Users shall bookmark/unbookmark favourite artists (`user_favourite_artists`); used as the M_{artist} signal in the recommendation engine.	M
FR-11	Preference Update	Users shall update any preference at any time; changes immediately influence future recommendations.	H

Module 3: Music Discovery & Catalogue

Req ID	Requirement	Description	Pri
FR-12	Catalogue Browsing	Users shall browse a paginated catalogue of songs, albums and artists.	H
FR-13	Search	Users shall search songs by title, artist or album. Database-backed searches shall meet the response-time target in NFR-P1; searches relayed to the iTunes Search API depend on that service's latency.	H
FR-14	Genre/Mood Filter	Users shall filter the catalogue by one or more genres and/or moods simultaneously.	M
FR-15	Audio Preview	Users shall play 30-second audio previews (iTunes preview URLs) with controls (play, pause, seek, volume) via a persistent mini-player across routes.	H
FR-16	Song Bookmarking	Users shall bookmark/unbookmark songs as favourites (`user_favourite_songs`).	M

Module 4: Recommendation Engine

Req ID	Requirement	Description	Pri
FR-17	Personalized Recommendations	The system shall generate up to 20 ranked song recommendations using $S(u,s) = 0.30\,M_{genre} + 0.20\,M_{mood} + 0.20\,M_{artist} + 0.15\,M_{era} + 0.15\,M_{history}$ (Section 3.3.1).	H
FR-18	Recommendation Transparency	Each recommended song shall display a human-readable explanation of why it was recommended (score breakdown by component).	M
FR-19	Recommendation Refresh	Users shall manually trigger a recommendation refresh re-computed from their latest preferences.	M
FR-20	Cold Start Handling	For a cold-start user (no listening history), the system shall compute $S(u,s)$ without $M_{history}$ and renormalise the remaining weights by $1/0.85$ as defined in Section 3.3.1, so that scores remain in $[0,1]$.	H

Module 5: Playlist Management

Req ID	Requirement	Description	Pri
FR-21	Playlist Creation	Users shall create named playlists (max 100 characters) with an optional description and cover image.	H
FR-22	Playlist Visibility	Users shall set playlist visibility as Public (discoverable) or Private (owner-only), stored in `playlists.is_public`.	M

Req ID	Requirement	Description	Pri
FR-23	Add Songs	Users shall add any catalogue song to any of their playlists.	H
FR-24	Reorder Songs	Users shall reorder songs within a playlist using drag-and-drop (`playlist_songs.position`).	M
FR-25	Remove Songs	Users shall remove individual songs from a playlist.	H
FR-26	Delete Playlist	Users shall delete a playlist after a confirmation dialog.	M
FR-27	View Public Playlists	Users shall view any public playlist shared by another user.	M

Module 6: Music Blend (Collaborative Feature) — Core Feature

Req ID	Requirement	Description	Pri
FR-28	Initiate Blend	Authenticated User A shall initiate a Blend. The system shall generate a unique 6-character alphanumeric invitation code and store the session in `blend_sessions`.	H
FR-29	Blend Code Sharing	User A shall copy or share the invite code via a shareable URL.	H
FR-30	Join Blend	User B shall join an active blend by entering the invite code. The system validates the code (existing, pending and not expired) and links both users.	H
FR-31	Compatibility Score	The system shall compute $C(u_A, u_B) \in [0\%, 100\%]$ using the Jaccard-based composite formula in Section 3.3.2.	H
FR-32	Blended Playlist	The system shall generate a 20-track playlist per Section 3.3.3: 8 (40%) for User A, 8 (40%) for User B, 4 (20%) mutual.	H
FR-33	Save Blend Result	The system shall allow saving the generated blended playlist to the user's library (`playlists`).	M
FR-34	Pre-built Blends	The system shall maintain 24+ curated duo blend presets for quick use.	L
FR-35	Blend History	Users shall view the history of their past blend sessions (`blend_sessions`) with compatibility scores.	L

Module 7: Social Features

Req ID	Requirement	Description	Pri
FR-36	Friend Connections	Users shall send and accept friend requests, stored in the `friendship` table.	M
FR-37	Song Snap Sharing	Users shall send any song as a Song Snap to a connected friend, with notification and 30-second preview.	M
FR-38	Social Feed	Users shall view a feed of recently shared songs and new public playlists from connected friends.	L

Module 8: Listening History & Streaks

Req ID	Requirement	Description	Pri
FR-39	History Logging	The system shall auto-log every song preview playback with a timestamp into `listening_history`.	H
FR-40	Streak Tracking	The system shall display the current consecutive listening streak (days), maintained by ≥ 1 play per day (`listening_streaks`).	M
FR-41	History-based Recommendations	Listening history shall feed the $M_{history}$ signal (15% weight) in the recommendation engine.	H

Module 9: Administration

Req ID	Requirement	Description	Pri
FR-42	Catalogue Management	Administrators shall add, edit and remove songs, albums and artists via an admin interface accessible only to the Administrator role.	M
FR-43	User Management	Administrators shall view and deactivate accounts violating platform policies. Privileged account operations execute server-side only.	L
FR-44	Content Enforcement	Administrators shall remove playlist content or Song Snaps reported as inappropriate.	L

Normal users have no access to admin interfaces or admin-only database operations; see NFR-S8 and NFR-S9.

3.3 Core Algorithm Specifications

The following notation is used identically in **FR-17**, **FR-31**, UC-02, UC-03 and Sections 3.3.1–3.3.3.

Symbol	Meaning
$S(u,s)$	Recommendation score of song s for user u, $S \in [0,1]$
M_{genre}, M_{mood}, M_{artist}, M_{era}, $M_{history}$	Component match scores (each in $[0,1]$) for genre, mood, artist, era and listening history
$C(u_A,u_B)$	Compatibility score of users A and B, $C \in [0\%,100\%]$
$J(X,Y)$	Jaccard similarity $\lvert X \cap Y\rvert / \lvert X \cup Y\rvert$ ($J=0$ when $X \cup Y$ is empty)
G_X, MD_X, E_X, H_X	Sets of user X: preferred genres, preferred moods, preferred eras (decades), listening history (distinct song IDs in `listening_history`)
Top50(u)	The 50 catalogue songs with the highest $S(u,s)$

3.3.1 Recommendation Score Formula

For a user u and song s, the recommendation score is

$$S(u,s) = 0.30\, M_{genre}(u,s) + 0.20\, M_{mood}(u,s) + 0.20\, M_{artist}(u,s) + 0.15\, M_{era}(u,s) + 0.15\, M_{history}(u,s). \tag{1}$$

Component	Weight	Computation	Range
M_{genre}	0.30	1 if the song's genre $\in$ the user's preferred genres, else 0	$[0,1]$
M_{mood}	0.20	1 if the song's mood $\in$ the user's preferred moods, else 0	$[0,1]$
M_{artist}	0.20	1 if the song's artist $\in$ the user's favourite artists, else 0	$[0,1]$
M_{era}	0.15	1 if the song's release decade $\in$ the user's preferred eras, else 0	$[0,1]$
$M_{history}$	0.15	Normalised play frequency $=$ plays(u,s)/maxplays(u), where plays(u,s) counts rows of `listening_history` and maxplays(u) is the highest play count over all songs for u	$[0,1]$

The weights sum to 1.0, so $S(u,s) \in [0,1]$. Songs are ranked in descending order of S (ties broken by song id for determinism).

Cold-start weighting (users with no listening history). A user has no listening history when maxplays$(u) = 0$. Then $M_{history}$ is undefined and is **excluded**; the remaining weights are divided by their sum so that they again add up to 1. Formally, with weights $w_k = (0.30, 0.20, 0.20, 0.15, 0.15)$ and active component set $K(u)$:

$$S(u,s) = \frac{\sum_{k \in K(u)} w_k \, M_k(u,s)}{\sum_{k \in K(u)} w_k}. \tag{2}$$

For a user with history, $K(u) = \{genre, mood, artist, era, history\}$ and the denominator is 1.00. For a cold-start user, $K(u) = \{genre, mood, artist, era\}$ and the denominator is $1 - 0.15 = 0.85$, giving the effective weights below (sum $= 1.0000$).

Component	Original weight	Cold-start weight $= w/0.85$
M_{genre}	0.30	$0.30/0.85 \approx 0.3529$
M_{mood}	0.20	$0.20/0.85 \approx 0.2353$
M_{artist}	0.20	$0.20/0.85 \approx 0.2353$
M_{era}	0.15	$0.15/0.85 \approx 0.1765$
$M_{history}$	0.15	excluded (weight 0)

If a cold-start user has bookmarked no artists, $M_{artist}(u,s) = 0$ for every song (the component stays active; no further redistribution is applied).

3.3.2 Taste Compatibility Formula

The Jaccard similarity index between two sets X and Y is

$$J(X,Y) = \frac{|X \cap Y|}{|X \cup Y|}. \tag{3}$$

The composite compatibility score between User A and User B is

$$\begin{aligned} C(u_A, u_B) = \Big[& 0.40\, J(G_A, G_B) + 0.30\, J(MD_A, MD_B) \\ & + 0.20\, J(E_A, E_B) + 0.10\, J(H_A, H_B) \Big] \times 100\% \end{aligned} \tag{4}$$

(genre overlap, mood overlap, era overlap, listening-history overlap). The output satisfies $C \in [0\%, 100\%]$, and the score is symmetric, $C(u_A, u_B) = C(u_B, u_A)$, because J is symmetric. **History**

term: if either user has an empty listening history, the history term is omitted and the remaining weights are divided by 0.90 (effective weights 0.4444 genre, 0.3333 mood, 0.2222 era), so the score still spans 0–100%.

3.3.3 Blended Playlist Generation

Segment	Count	Percentage	Source
Mutual discovery tracks	4	20%	Songs in Top50$(u_A) \cap$ Top50(u_B), ordered by $S_{joint}(s) = \big(S(u_A, s) + S(u_B, s)\big)/2$; if fewer than 4 exist, remaining slots are filled by the next-highest S_{joint} songs
User A profile tracks	8	40%	Highest-ranked songs from $S(u_A, s)$ not already selected
User B profile tracks	8	40%	Highest-ranked songs from $S(u_B, s)$ not already selected

The segments are filled in the order mutual → User A → User B and duplicates are removed, so a song appears only once and the total remains 20 (8 + 8 + 4) whenever the catalogue contains enough songs. If a mutual track would also appear in a user's top-8, it is counted in the mutual segment and that user's segment is back-filled with the next-ranked song.

4 Other Nonfunctional Requirements

Status convention. Security, usability, maintainability and design-rule requirements are mandatory design requirements. Performance and reliability figures (NFR-P, NFR-R4) are **prototype evaluation targets**: they define how the prototype will be measured, not guaranteed production capabilities. Scalability items (NFR-SC) are **architectural goals / future considerations** that have not been implemented or load-tested in the prototype.

4.1 Performance Requirements

Req ID	Target	Verification (planned)
NFR-P1	Catalogue search and recommendation queries served by the Supabase/PostgreSQL backend return within 300 ms for 95% of requests under prototype test conditions. Latency of requests relayed to external APIs (iTunes, Spotify) depends on those services and is excluded.	Browser timing; k6
NFR-P2	Music Blend compatibility score computation completes within 1 second for any user pair.	Unit/timing test
NFR-P3	Evaluation target: handle 500 concurrent authenticated sessions without noticeable degradation, subject to Supabase and Vercel plan quotas. Not yet verified at the prototype stage.	k6 load test (not yet executed)
NFR-P4	Audio preview files shall be streamed directly from the iTunes-provided CDN-hosted preview URLs (MelodyBlend does not host audio) so that 30-second previews buffer smoothly.	Manual playback check

Req ID	Target	Verification (planned)
NFR-P5	Target: First Contentful Paint under 2 seconds on a 10 Mbps connection for the landing and dashboard pages.	Lighthouse
NFR-P6	Database queries shall use indexed foreign keys on all join columns to avoid full-table scans.	Schema review; EXPLAIN

4.2 Security Requirements

No one other than a registered user can enter the protected parts of the website, and each user class can perform only its own actions.

Req ID	Requirement
NFR-S1	Password handling is delegated to Supabase Auth. Passwords are securely handled by Supabase Auth, and plaintext passwords are never stored or logged by the MelodyBlend application.
NFR-S2	All client-server communication shall occur exclusively over HTTPS/TLS 1.2+ (enforced by the Vercel and Supabase endpoints).
NFR-S3	Authentication shall use signed JWT access tokens issued by Supabase Auth; the application does not implement its own token signing. Default maximum access-token lifetime is 1 hour with silent refresh.
NFR-S4	All user-specific tables (`profiles`, `user_preferences`, `user_favourite_songs`, `user_favourite_artists`, `playlists`, `playlist_songs`, `listening_history`, `listening_streaks`, `friendship`, `song_snap`, `blend_sessions`) shall enforce PostgreSQL RLS policies based on the authenticated user (`auth.uid()`). Catalogue tables (`songs`, `albums`, `artists`, `genres`, `moods`) are read-only to normal users.
NFR-S5	The Supabase service-role key (and the Spotify client secret, if used) shall be stored only in server-side environment variables, never prefixed for client exposure, and shall never appear in client-side JavaScript bundles.
NFR-S6	State-changing Next.js Route Handlers / Server Actions that rely on cookie-based sessions shall be protected against CSRF using SameSite cookies and Origin verification. Direct Supabase API calls use bearer tokens rather than ambient cookies.
NFR-S7	Input shall be validated client-side (TypeScript) for usability and server-side in Route Handlers and database constraints (types, NOT NULL, CHECK, foreign keys). SQL injection is mitigated by parameterised queries through the Supabase client/PostgREST; XSS is mitigated by React output escaping. RLS enforces authorisation, not input validation.
NFR-S8	The Administrator role shall be distinct from normal users and held in a server-controlled field (Supabase Auth `app_metadata`) that users cannot edit. Admin screens, admin route handlers and write policies on catalogue tables shall check this role.
NFR-S9	Admin operations needing elevated privileges (e.g., deactivating accounts, deleting auth users) shall execute only in server-side code after verifying the Administrator role; service-role credentials shall never be exposed to the front end.

4.3 Software Quality Attributes

Testing and review with users and the supervisor continue throughout the development phase so that quality is maintained and all requirements are fulfilled. Database, logical and UI testing are

required (Section 6).

4.3.1 Usability (NFR-U)

Req ID	Requirement
NFR-U1	The UI shall adopt a glassmorphic dark-mode aesthetic with micro-animations on interactive elements.
NFR-U2	The platform shall be fully responsive on desktop ($\geq$1024 px), tablet (768–1023 px) and mobile (320–767 px).
NFR-U3	Target: all primary user journeys are completable in 5 or fewer user interactions.
NFR-U4	Error messages shall be descriptive and actionable. Generic server errors shall not be exposed to users.
NFR-U5	Target: keyboard navigation for all primary elements (WCAG 2.1 Level A).
NFR-U6	All loading states shall display skeleton screens or progress indicators within 100 ms of initiating a fetch.

4.3.2 Reliability (NFR-R)

Req ID	Requirement
NFR-R1	Recommendation score $S(u, s)$ shall be deterministic and idempotent: identical inputs always produce identical ranked outputs.
NFR-R2	Compatibility score $C(u_A, u_B)$ shall be symmetric: $C(u_A, u_B) = C(u_B, u_A)$.
NFR-R3	The system shall provide a read-only fallback seed dataset for discovery/recommendation when Supabase is unreachable (demonstration resilience; user data is not persisted in fallback mode).
NFR-R4	Evaluation target: 99.5% availability of the Supabase-hosted backend during the prototype evaluation period. Actual availability depends on Supabase and Vercel service levels; no independent SLA is assumed by this project.
NFR-R5	The system shall handle API timeouts with a maximum of 3 retries using exponential backoff before displaying an error.

4.3.3 Maintainability (NFR-M)

Req ID	Requirement
NFR-M1	The codebase shall be written entirely in TypeScript with strict mode enabled.
NFR-M2	The front end shall use React functional components and hooks exclusively (no class components).
NFR-M3	Utility functions (scoring, Jaccard, blend generation) shall be isolated in separate, unit-testable modules.
NFR-M4	The database schema shall be maintained using version-controlled SQL migration files.
NFR-M5	Target: at least 60% test coverage on algorithmic utility functions.

4.3.4 Scalability (NFR-SC) — Architectural Goals / Future Considerations

*The following are design goals that guide the architecture. They are **not** implemented or load-tested in the prototype, and the prototype does not claim to achieve these figures.*

Req ID	Goal	Status
NFR-SC1	The PostgreSQL schema should allow `listening_history` to be partitioned (e.g., by `played_at` range) if logging volume grows.	Future consideration
NFR-SC2	The recommendation engine is designed to be stateless so it can run in parallel across serverless invocations.	Design goal (not load-tested)
NFR-SC3	The Realtime channel architecture should be able to scale toward 1,000 concurrent blend sessions, subject to Supabase plan limits.	Goal; not demonstrated

4.4 Business Rules

- MelodyBlend plays **only** 30-second iTunes previews; full-length streaming is not offered and the Spotify Web API is never used for audio.
- A Blend involves exactly **two** registered users, joined through a unique 6-character invitation code.
- A blended playlist contains 20 tracks: 8 from User A's profile, 8 from User B's profile and 4 mutual tracks.
- Only registered, authenticated users can use personalised features; guests may only view the landing, login and registration pages.
- A user can read and change only their own private data (enforced by RLS); private playlists are never visible to other users.
- Only users holding the Administrator role can modify the music catalogue, deactivate accounts or remove reported content.
- Deleting an account removes all personal data owned by that user (cascade delete).
- The platform shall comply with the terms of the iTunes Search API and the Spotify Web API, including rate limits.

4.5 External Interface Requirements

User interface: responsive web UI (Section 4, NFR-U). **Hardware interfaces:** none beyond a standard web-capable device with audio output.

Interface	Protocol	Format	Purpose
Supabase Auth API	HTTPS/REST	JSON	User registration, login, token refresh, logout, password reset
Supabase Database (PostgREST)	HTTPS/REST	JSON	CRUD operations on database tables (RLS enforced)
Supabase Realtime	WebSocket	JSON	Real-time blend session notifications
Supabase Storage	HTTPS/REST	Binary/JSON	Avatar and playlist cover images
iTunes Search API	HTTPS/REST	JSON	Catalogue metadata, album artwork, 30-second preview URLs
Spotify Web API	HTTPS/REST	JSON	Additional artist/track metadata enrichment only (no streaming)
Email service (Supabase SMTP)	SMTP	Email	Verification and password-reset emails

4.6 Role of the External Music APIs

API	Used for	Not used for
iTunes Search API (primary)	Catalogue metadata (title, artist, album, genre, release year), artwork, and 30-second preview URLs used by the audio player.	Full-length playback; user authentication.
Spotify Web API (enrichment only)	Additional artist/track metadata that supplements the catalogue. Optional: if unavailable, all core features continue to work using iTunes and database data.	Audio playback or streaming of any kind; user login or playlist sync with Spotify accounts.

MelodyBlend does not provide full Spotify streaming. All audio played in the platform is a 30-second iTunes preview. Credentials for the Spotify Web API, if used, are held server-side.

4.7 Design and Implementation Constraints

- **External API limits:** the iTunes Search API and the Spotify Web API may throttle requests (e.g., HTTP 429); the system retries with exponential backoff (NFR-R5).
- **Audio preview restriction:** playback is limited to 30-second previews supplied by the iTunes Search API due to licensing constraints.
- **Duo blend limit:** the blend algorithm supports exactly 2 users; a multi-user extension requires architectural changes.
- **Free-tier quotas:** Supabase and Vercel free tiers limit storage, bandwidth, real-time connections and function usage; any production use requires paid plans.
- **JWT expiry:** Supabase access tokens have a default lifetime of 1 hour (configurable); the Supabase client refreshes them silently.
- **RLS enforcement:** all client mutations are subject to PostgreSQL RLS. Privileged (admin) operations run only in server-side code using the service-role key, which is never shipped to the browser.
- **Secrets:** the Spotify client secret and the Supabase service-role key are held in server-side environment variables only.
- **Browser autoplay policy:** user interaction is required before the first audio play event.
- **TypeScript strict mode:** `strict: true` is enforced — no implicit `any`, full null-safety required.

4.8 Assumptions and Dependencies

Assumptions:

- Users have a stable internet connection for discovery and audio preview.
- The music catalogue is pre-populated with sufficient seed data for recommendation evaluation.
- Both users participating in a Blend are registered and authenticated.
- The iTunes Search API remains publicly available and continues to return 30-second preview URLs.

Key dependencies (versions kept broad; exact versions are pinned in the project lock file):

Dependency	Version / tier	Purpose
Next.js	15+ (App Router)	Front-end framework: SSR/CSR, Route Handlers, built-in dev/build toolchain
React	19	Component-based UI library
TypeScript	5.x (strict mode)	Static typing for the entire codebase
Tailwind CSS	3.x or later	Utility-first responsive UI styling
Lucide React	Latest	Vector icon set
Supabase (Auth, PostgREST, Realtime, Storage) and client libraries	Free / Pro tier	Authentication, database API, real-time notifications, file storage
PostgreSQL	15+ (as provisioned by Supabase)	Relational database with Row Level Security
iTunes Search API	Public (no API key)	Catalogue metadata, album artwork, 30-second preview URLs
Spotify Web API	Free developer tier	Additional artist/track metadata enrichment only (optional)
Vercel	Hobby / Pro tier	Deployment of the Next.js application
Node.js / npm	$\geq$ 18.18 LTS	Development runtime and package manager
Jest, React Testing Library	Latest	Unit and component tests for algorithm modules and UI
Playwright, Lighthouse, k6 (planned)	Latest	End-to-end, front-end performance and load evaluation (not yet executed)

5 Other Requirements

- **Maintenance.** MelodyBlend needs ongoing maintenance as a long-lived software product. It will require refactoring, and requirements may change as music APIs, platform quotas and user expectations evolve. Schema changes are made only through version-controlled SQL migration files (NFR-M4).
- **Legal and licensing.** The platform shall respect the licensing limits of the music data providers: only 30-second previews are played, and API rate limits and terms of use are observed.
- **Data protection.** Personal data is protected by RLS and Supabase Auth; a user can request deletion of their account together with all of their personal data (**FR-06**).
- **Future work.** Planned extensions (not part of this release) are full-length licensed streaming, multi-user ($>$ 2) blends and native mobile applications.

6 Verification and Validation

6.1 Testing Strategy

Testing level	Scope	Tools	Status
Unit testing	Isolated algorithmic functions (Jaccard similarity, recommendation scoring, cold-start weighting, blend partitioning)	Jest	Automated suite planned; core flows verified manually
Integration testing	Supabase Auth ↔ PostgreSQL ↔ React component data flows	Testing Library	Planned
Functional (E2E) testing	Full user journeys: registration, discovery, blend creation	Playwright	Planned (manual walkthroughs done for TC-01 – TC-05)
Security testing	RLS policy enforcement, JWT validation, XSS/SQL injection resistance	Manual + Supabase policy tests	Planned
Performance testing	Response times for search/recommendation queries; load behaviour (targets in Section 4)	Lighthouse, k6	Planned; not yet executed

6.2 Initial Test Cases

*Status legend: **PASS** = the feature is implemented in the prototype and the test was executed manually with the expected result observed; **PLANNED** = test designed but not yet executed. No test is reported as PASS unless it has been executed.*

Test ID	Scenario	Expected result	Related req.	Status
TC-01	Valid user login	Authentication succeeds; redirected to Dashboard	**FR-02**, **FR-03**	PASS
TC-02	Invalid credentials	Login rejected; descriptive error displayed	**FR-02**, NFR-U4	PASS
TC-03	Preference submission	Genres and moods stored in `user_preferences`; recommendation output changes	**FR-07**–**FR-09**, **FR-11**	PASS
TC-04	Recommendation scoring	Up to 20 songs returned in descending $S(u,s)$ order with explanations	**FR-17**, **FR-18**, **FR-41**	PASS
TC-05	Duo blend calculation	Compatibility score and blended playlist generated	**FR-28**–**FR-32**	PASS
TC-06	Unauthorised RLS access	RLS policy blocks the mutation: request rejected or zero rows affected; data unchanged	NFR-S4	PLANNED
TC-07	Cold-start recommendation	Recommendations generated without $M_{history}$ using weights renormalised by $1/0.85$; no null-pointer errors	**FR-20**	PLANNED

Test ID	Scenario	Expected result	Related req.	Status
TC-08	Invalid blend code	System displays "Invalid or expired invitation code"	**FR-30**	PLANNED
TC-09	Password reset flow	Reset email delivered; user can set a new password	**FR-05**	PLANNED
TC-10	Recommendation idempotency	Identical ranked output for the same user with the same preferences (run twice)	**FR-17**, NFR-R1	PLANNED
TC-11	Compatibility symmetry	$C(u_A, u_B) = C(u_B, u_A)$ confirmed numerically	**FR-31**, NFR-R2	PLANNED
TC-12	Playlist visibility	Private playlist not visible to other authenticated users via the API	**FR-22**, **FR-27**	PLANNED

Conformance of the 8-8-4 blend partition (Section 3.3.3) is covered by the planned unit-test suite. Performance and scalability targets are evaluated separately (Section 4) and are not yet executed.

A Technology Stack Summary

Layer	Technology	Version	Purpose
Front-end framework	Next.js + React + TypeScript	15+ / 19 / 5.x	Web application (App Router, SSR/CSR, Route Handlers)
Styling	Tailwind CSS	3.x or later	Responsive dark-mode UI
Icons	Lucide React	Latest	Vector icons
Backend, Auth, Realtime	Supabase	Free / Pro tier	Auth (JWT sessions), PostgREST, Realtime, Storage
Database	PostgreSQL	15+ (via Supabase)	Relational schema with RLS
Music catalogue and preview	iTunes Search API	Public	Catalogue metadata, album artwork, 30-second preview URLs
Metadata enrichment	Spotify Web API	Developer tier	Additional artist/track metadata only; no streaming
Build and dev tooling	Next.js toolchain (`next dev` / `next build`) + npm + TypeScript compiler	Latest	Development server, compilation, bundling, type-checking
Testing	Jest, React Testing Library; Playwright, Lighthouse, k6 (planned)	Latest	Unit/component tests; E2E, performance and load evaluation
Deployment	Vercel	Hobby / Pro tier	Hosting of the Next.js application and serverless functions

B Development Phase Plan

Phase	Timeline	Deliverables	Assigned
Phase 1	Weeks 1–3	Database schema, authentication, catalogue setup	Prashant / Pranav
Phase 2	Weeks 4–6	Recommendation engine, preference profiling UI	Pridhi / Hazel
Phase 3	Weeks 7–9	Music Blend algorithm, invite codes, real-time notifications	Prashant / Hazel
Phase 4	Weeks 10–12	UI refinement, testing, evaluation, deployment	Hazel / Pranav

C Team Responsibilities

Member	Roll No.	Responsibilities
Prashant Narayan Verma	1024160140	System architecture, database modelling, backend integration
Hazel Latka	1024160105	Front-end UI/UX design, component development, responsiveness
Pridhi Thareja	1024160136	Recommendation algorithms, compatibility calculation, playlist blending
Pranav Chauhan	1024160113	Quality assurance, test automation, system evaluation

D Known Limitations

- **30-second preview only:** audio playback is limited to the 30-second preview clips provided by the iTunes Search API due to music licensing constraints. MelodyBlend does not provide full-length or Spotify streaming.
- **Spotify role:** the Spotify Web API is used only for optional artist/track metadata enrichment, never for audio.
- **Duo blend only:** the current blend algorithm supports exactly 2 users. Group blending (> 2) requires architectural extension.
- **Offline mode:** offline functionality relies on a read-only seed dataset; full offline support is not guaranteed.
- **API rate limits:** heavy usage may trigger iTunes/Spotify throttling (e.g., HTTP 429); backoff is implemented but may cause momentary delays.
- **Unverified targets:** performance and availability figures (NFR-P, NFR-R4) are prototype evaluation targets and have not yet been load-tested; they are not production guarantees.
- **Scalability goals:** partitioning of `listening_history` and support for 1,000 concurrent blend sessions (NFR-SC) are architectural goals for future work, not implemented or demonstrated.
- **Testing maturity:** only TC-01 – TC-05 have been executed (manually); automated unit, integration, E2E, security and performance suites remain planned.

E Requirements Traceability Matrix

Each functional requirement is linked to its module, use case, DFD process, data store (table) and test case where applicable ("—" = no dedicated prototype test case yet).

FR	Module	Use case	DFD	Data store / component	Test
FR-01	M1	UC-01	P1	`auth.users`, `profiles`	—
FR-02	M1	UC-04	P1	`auth.users` (Supabase Auth)	TC-01, TC-02
FR-03	M1	UC-04	P1	Supabase session / JWT	TC-01
FR-04	M1	UC-04	P1	`profiles`, Supabase Storage	—
FR-05	M1	UC-04	P1	`auth.users`, SMTP	TC-09
FR-06	M1	UC-04	P1	`profiles` (cascade), server route	—
FR-07	M2	UC-05, UC-01	P2	`genres`, `user_preferences`	TC-03
FR-08	M2	UC-05, UC-01	P2	`moods`, `user_preferences`	TC-03
FR-09	M2	UC-05, UC-01	P2	`user_preferences` (era)	TC-03
FR-10	M2	UC-05	P2	`user_favourite_artists`	—
FR-11	M2	UC-05	P2	`user_preferences`	TC-03
FR-12	M3	UC-06	P3	`songs`, `albums`, `artists`	—
FR-13	M3	UC-06	P3	`songs`, `albums`, `artists`; iTunes API	—
FR-14	M3	UC-06	P3	`genres`, `moods`, `songs`	—
FR-15	M3	UC-06	P3	`songs.preview_url`; iTunes previews	—
FR-16	M3	UC-06	P3	`user_favourite_songs`	—
FR-17	M4	UC-03	P4	`songs`, `user_preferences`, `listening_history`	TC-04, TC-10
FR-18	M4	UC-03	P4	recommendation module	TC-04
FR-19	M4	UC-03	P4	recommendation module	—
FR-20	M4	UC-03	P4	recommendation module (Sec. 3.3.1)	TC-07
FR-21	M5	UC-07	P5	`playlists`	—
FR-22	M5	UC-07	P5	`playlists.is_public`	TC-12
FR-23	M5	UC-07	P5	`playlist_songs`	—
FR-24	M5	UC-07	P5	`playlist_songs.position`	—
FR-25	M5	UC-07	P5	`playlist_songs`	—
FR-26	M5	UC-07	P5	`playlists`	—
FR-27	M5	UC-07	P5	`playlists`	TC-12
FR-28	M6	UC-02	P6	`blend_sessions`	TC-05
FR-29	M6	UC-02	P6	`blend_sessions.invite_code`	—
FR-30	M6	UC-02	P6	`blend_sessions`; Realtime	TC-08
FR-31	M6	UC-02	P6	`user_preferences`, `listening_history`	TC-05, TC-11

FR	Module	Use case	DFD	Data store / component	Test
FR-32	M6	UC-02	P6	`songs`, `playlists`	TC-05
FR-33	M6	UC-02	P6/P5	`playlists`	—
FR-34	M6	UC-02 (A3)	P6	`playlists` (curated presets)	—
FR-35	M6	UC-02 (A4)	P6	`blend_sessions`	—
FR-36	M7	UC-08	P7	`friendship`	—
FR-37	M7	UC-08	P7	`song_snap`	—
FR-38	M7	UC-08	P7	`song_snap`, `playlists`, `friendship`	—
FR-39	M8	UC-09	P8	`listening_history`	—
FR-40	M8	UC-09	P8	`listening_streaks`	—
FR-41	M8	UC-09, UC-03	P8→P4	`listening_history`	TC-04
FR-42	M9	UC-10	P9	`songs`, `albums`, `artists`, `genres`, `moods`	—
FR-43	M9	UC-10	P9	`auth.users`, `profiles` (server-side)	—
FR-44	M9	UC-10	P9	`playlists`, `playlist_songs`, `song_snap`	—

Non-functional traceability: NFR-S4 → TC-06; NFR-R1 → TC-10; NFR-R2 → TC-11; NFR-U4 → TC-02; NFR-P1 – NFR-P6 and NFR-R4 → the performance evaluation plan in Section 6 (not yet executed).

End of Software Requirements Specification — MelodyBlend v1.1

Prepared by: Prashant Narayan Verma ● Hazel Latka ● Pridhi Thareja ● Pranav Chauhan
Course: UCS503 — Software Engineering | TIET Patiala | October 2026